

COMPSCI 589 Homework 3 - Spring 2026

Due **March 30**, 11:55 pm Eastern Time

Instructions

- This homework assignment consists of a programming part.
- While you may discuss problems with your peers (e.g., to discuss high-level approaches), you must answer the questions on your own and implement all solutions independently. In your submission, do explicitly list all students with whom you discussed this assignment.
- Submissions must be typed. You must use $\text{\LaTeX}$ to prepare your submission. Handwritten and scanned submissions will not be accepted.
- The written report, including your answers, plots, and analysis, should be submitted on Gradescope as a PDF via the appropriate submission link. The source code should be submitted as a single .zip file through the separate programming assignment link on Gradescope.
- We strongly encourage using Python 3 for your homework code. You may use other languages. In either case, you *must* provide clear instructions for running your code and reproducing your experiments.
- You *may not* use any machine learning-specific libraries in your code, e.g., TensorFlow, PyTorch, or machine learning algorithms implemented in scikit-learn (though you *may* use other functions provided by this library, such as one that splits a dataset into training and testing sets). You may also use libraries such as numpy and matplotlib. If you are not certain whether a specific library is allowed, do ask us.
- All submissions will be checked for plagiarism using two independent plagiarism-detection tools. Renaming variable or function names, moving code within a file, etc., are all strategies that *do not* fool the plagiarism-detection tools we use. Your code will also be compared with submissions from your classmates and with publicly available implementations online.
- If you get caught, all penalties mentioned in the syllabus *will* be applied—which may include directly failing the course with a letter grade of “F”.

→ Before starting this homework, please review this course’s policies on plagiarism by reading the corresponding section of the syllabus.

- The automated system will not accept assignments after 11:55 pm on March 30.
- The TeX file for this homework (which you should use to write your solutions in $\text{\LaTeX}$), as well as the datasets you will investigate, are available on Canvas under “Homework #3 - Supporting Files”. See below for more details.

Programming Assignment [100 Points, total]

In this homework, you will be implementing the Random Forest algorithm. **Notice that you may not use existing machine learning code for this problem: you must implement the learning algorithms entirely on your own and from scratch.**

In this assignment, you will:

- Leverage your implementation of the standard Decision Tree algorithm, based on the Information Gain splitting criterion, to implement a Random Forest classifier.

Important: You must reuse *your* previous implementation of the Decision Tree algorithm.

Important: Your implementation of the Decision Tree algorithm should support both categorical *and* numerical attributes. When determining the numerical threshold to use when splitting numerical nodes, you may (i) use the average of that attribute's values in the partition of instances associated with the current node; or (ii) use the more sophisticated (and “powerful”) technique discussed in Lecture 05.

- Implement a procedure to create bootstrap datasets by sampling (*with replacement*) from the current training set to construct the datasets based on which each tree in the random forest will be trained.
- Implement a mechanism to select (whenever a node is being split) m attributes at random from the complete set of attributes of an instance. You should then split the node based on the attribute with the best Information Gain. As a heuristic, we usually use $m \approx \sqrt{X}$, where X is the total number of attributes in each instance of the dataset.
- Implement a majority voting mechanism that takes into account the predictions made by each tree in the ensemble to classify new instances.
- Implement the *stratified cross-validation* technique to evaluate the generalization capability of the random forest. For an example of how this technique works, see Figure 1. We recommend using $k = 5$ folds. Of these, $k - 1$ folds will be used to train a random forest composed of $ntree$ trees. Here, $ntree$ **will be the main adjustable hyperparameter of your algorithm.**

Note: In cross-validation, the remaining fold (the one not used during training) will be used to evaluate the performance of the random forest itself (i.e., not the performance of any individual tree separately!).

Another way to describe how cross-validation works in our setting is as follows: at each iteration of the cross-validation process, you will train not only *one* decision tree, but multiple ($ntree$) decision trees, all of which will be combined to form your random forest. You will then repeat this process, training and evaluating k random forests (one for each of the k cross-validation iterations). The final cross-validation performance you should return is the average of the performances computed for each of the folds.

- Evaluate the impact the number of trees (i.e., the $ntree$ hyperparameter) has on the performance of your random forest.

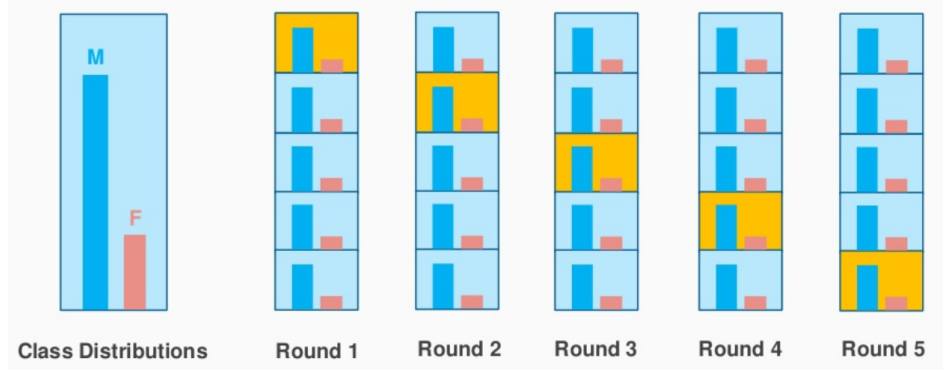


Figure 1: Example of how to construct *stratified* folds to perform k -fold cross-validation. Here, we assume two classes: **M** and **F**. Notice how the complete dataset (shown in the leftmost figure) is not balanced: approximately $3/4$ of its instances belong to the class **M**, and approximately $1/4$ of its instances belong to the class **F**. We wish to approximately preserve these proportions (i.e., the proportion of instances of class **M** and the proportion of instances of class **F**) in each of the $k = 5$ folds. To do this, you should first split the original dataset, D , into two: one containing only instances of class **M** (sub-dataset D_M), and another one containing only instances of class **F** (sub-dataset D_F). Then, given the total number of instances in the original dataset (D) and the number of folds you are using (k), you should be able to determine the size of each fold. Let us assume that each fold, in this example, should have 50 instances. To construct each fold, you should draw random instances from D_M and D_F . In this example, $3/4$ of the instances that you add to the current fold should come from D_M , and $1/4$ of the instances that you add to the current fold should come from D_F . Notice that because the folds should be disjoint (i.e., an instance should appear in only one fold), after drawing an instance either from D_M or D_F , it should be removed from such a set to ensure that it will not be accidentally included as part of another fold. (Source: [Stack Exchange](#)).

1 Deliverables and Datasets

The main objective of this homework is to study the impact of the number of trees in a random forest (i.e., the *n_{tree}* hyperparameter) on its performance. **When constructing each tree in the ensemble, you are free to choose among several stopping criteria.** You may, for instance:

- Stop splitting nodes whenever the data partition associated with the current node contains fewer than *n* instances, for some value of *n* that you choose empirically. We will refer to this as the `minimum_size_for_split` criterion.
- Stop splitting nodes whenever the information gain resulting from a split is not sufficiently high (even if it is slightly greater than zero). You are free to choose empirically the minimum information gain you would like to use. We will refer to this as the `minimum_gain` criterion.
- You may stop splitting whenever the current branch of the tree becomes too deep, that is, when its depth exceeds some pre-defined value that you select empirically. We will refer to this as the `maximum_depth` stopping criterion.

Important: In all questions below, you should explicitly mention which criterion or criteria you used and the corresponding hyperparameter(s). You should try different options to identify the setting that produces the best possible performance for your algorithm.

In this homework, you will be analyzing two datasets:

(1) **The Wisconsin Breast Cancer Diagnostic (WDBC) Dataset**. The goal here is to train a classifier capable of predicting whether a breast mass is malignant or benign, based on features computed from a digitized image of a breast mass. The dataset is composed of 569 instances. Each instance is described by 30 *numerical* attributes, and there are two classes.

(2) **The Loan Eligibility Prediction Dataset**. Here, the goal is to automatically (and accurately) predict whether a given person should qualify for a loan. Each instance is described by 11 attributes, including the gender of the applicant, whether the applicant is married, their income, information about their credit history, etc. The binary target class to be predicted is whether a given applicant's request for a loan was approved or not. Notice that this dataset contains 7 *categorical* attributes and 4 *numerical* attributes. There is a total of 480 instances in the dataset.

Note: The datasets above can be found in the “*Supporting Files*” zip file for this homework on Canvas.

2 Questions

For each dataset, you should:

1. Train a random forest and evaluate it using the stratified cross-validation technique discussed above. You should train random forests with different values of the *ntree* parameter: 1, 5, 10, 20, 30, 40, and 50.
2. For each value of *ntree*, you should measure the accuracy, precision, recall, and F1 score of the resulting random forest.

WDBC Hyperparameter ($k = 5$, max_depth=8, min_samples_split=2)

Loan Hyperparameter ($k = 5$, max_depth=5, min_samples_split=2)

Table 1: WDBC Result ($k = 5$, max_depth=8, min_samples_split=2)

ntrees	acc	prec	recall	f1
1	0.913921	0.928642	0.935563	0.931813
5	0.941993	0.942741	0.966354	0.954341
10	0.950720	0.953431	0.969131	0.961043
20	0.957768	0.957035	0.977543	0.966867
30	0.971835	0.970122	0.985955	0.977832
40	0.968357	0.964898	0.985955	0.975148
50	0.964848	0.959446	0.985955	0.972408

Table 2: Loan Result ($k = 5$, max_depth=5, min_samples_split=2)

ntrees	acc	prec	recall	f1
1	0.731248	0.773090	0.867662	0.816866
5	0.803956	0.791307	0.975893	0.873558
10	0.804021	0.793959	0.969878	0.872832
20	0.797770	0.791954	0.963953	0.868734
30	0.801850	0.791301	0.972908	0.872178
40	0.801894	0.792679	0.969923	0.871776
50	0.803956	0.793142	0.972908	0.873284

3. You should then create, for each dataset and for each of the four metrics described above, a graph that shows the value of *ntree* on the x-axis and the value of the corresponding metric on the y-axis. For example, you should include one graph showing the accuracy of the random forest as a function of *ntree* for Dataset #1, another graph showing the precision of the random forest as a function of *ntree* for Dataset #1, and similar graphs for the other metrics and for Dataset #2.

Important: The analyses mentioned above should result in a total of **8 graphs**. Please indicate clearly in the caption of each figure/graph which results are shown in that graph.

WDBC Hyperparameter ($k = 5$, max_depth=8, min_samples_split=2)

Loan Hyperparameter ($k = 5$, max_depth=5, min_samples_split=2)

WDBC:

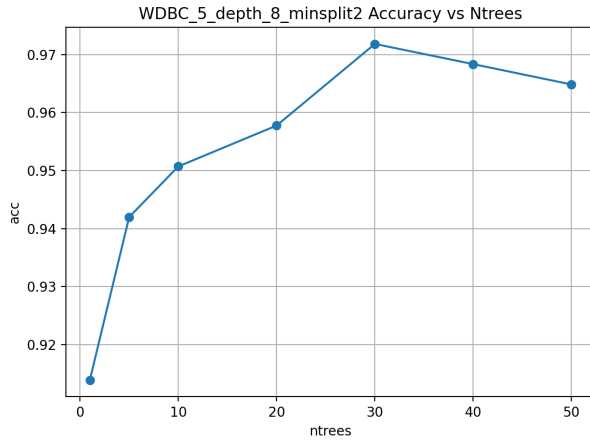


Figure 2: Accuracy for WDBC dataset.

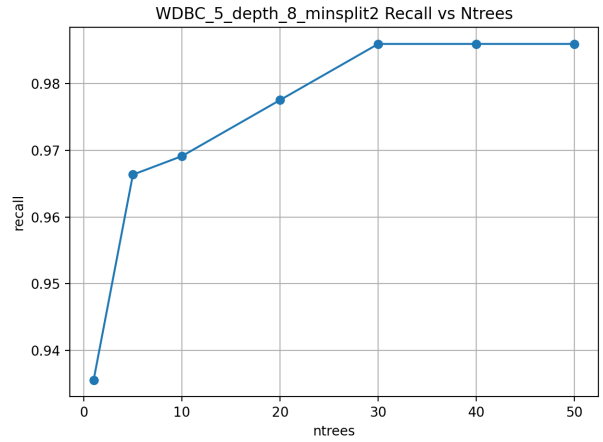


Figure 3: Recall for WDBC dataset.

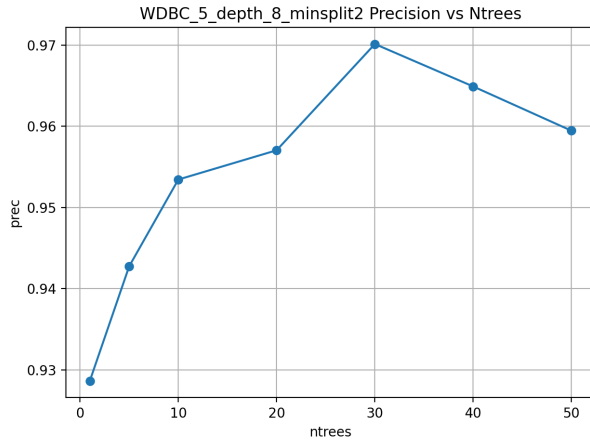


Figure 4: Precision for WDBC dataset.

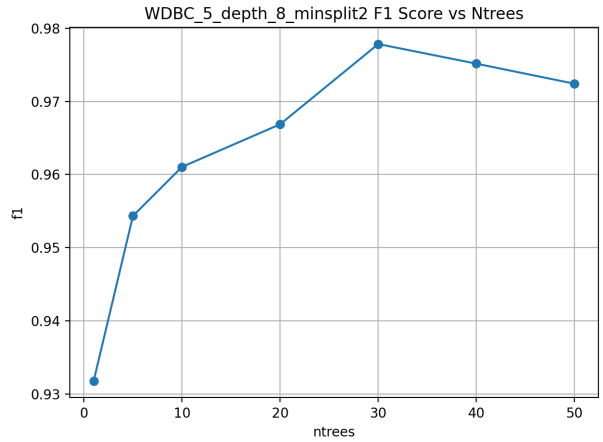


Figure 5: F1 for WDBC dataset.

Loan:

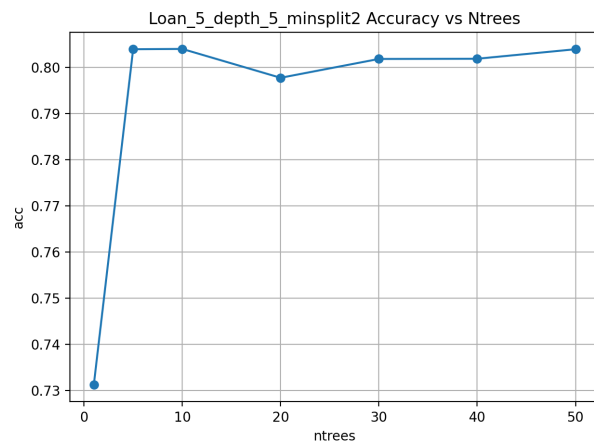


Figure 6: Accuracy for Loan dataset.

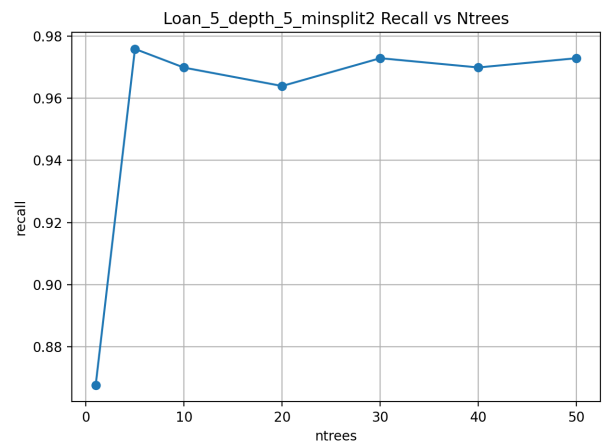


Figure 7: Recall for Loan dataset.

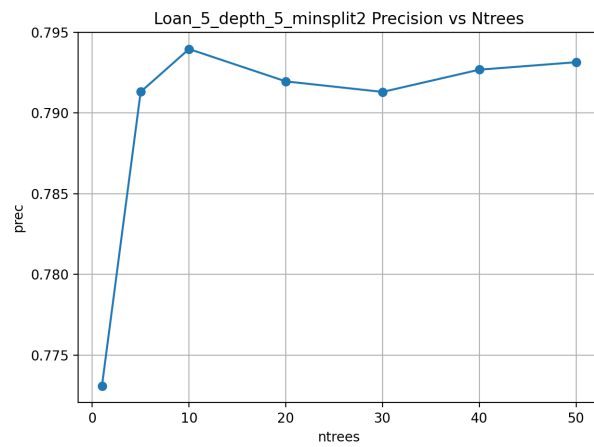


Figure 8: Precision for Loan dataset.

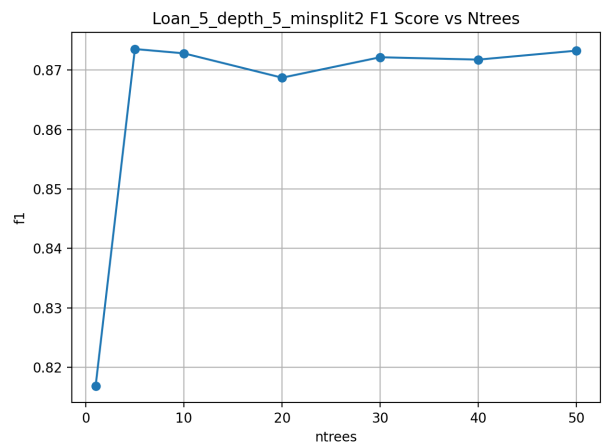


Figure 9: F1 for Loan dataset.

4. For each metric being evaluated (and for each dataset), discuss which value of *ntree* you would select if you were to deploy this classifier in real life. Explain your reasoning.

WDBC Hyperparameter ($k = 5$, max_depth=8, min_samples_split=2)

Loan Hyperparameter ($k = 5$, max_depth=5, min_samples_split=2)

WDBC ntree:

- Accuracy: 30
because it gives the highest accuracy.
- Precision: 30
because it gives the highest precision.
- Recall: 30/40/50
because these three values achieve the same highest recall.
- F1: 30
because it gives the highest f1.

Loan ntree:

- Accuracy: 10
because it gives the highest accuracy.
- Precision: 10
because it gives the highest precision.
- Recall: 5
because it gives the highest recall.
- F1: 5
because it gives the highest f1.

5. Discuss, at a high level, which metrics were more directly affected by changes in the value of *ntree* and, more generally, how these changes influenced the performance of your algorithm. For example, was the accuracy of the random forest particularly sensitive to increasing *ntree* beyond a certain value? Was the F1 score a “harder” metric to optimize, possibly requiring a larger number of trees in the ensemble? Is there a point beyond which adding more trees does not improve performance (or even makes it worse)?

For WDBC, all four metrics increase as *ntree* increases, showing a monotonic rise from 1 to 30: accuracy improves from 0.913921 to 0.971835, precision from 0.928642 to 0.970122, recall from 0.935563 to 0.985955, and F1 from 0.931813 to 0.977832. However, when *ntree* was further increased from 30 to 40 and 50, accuracy, precision, and F1 all decreased slightly, while recall remained at its peak value. On the WDBC dataset, accuracy, precision, and F1 are more sensitive to changes in *ntree*, whereas recall quickly reaches a plateau after reaching a high level. The optimal *ntree* hyperparameter for WDBC is 30; values above this do not yield further gains and may even result in a slight decline.

Loan improved significantly as *ntree* increased from 1 to 5, but after that, most metrics fluctuated only within a narrow range. Accuracy fluctuated between 0.797770 and 0.804021; precision fluctuated between 0.791 and 0.794; F1 fluctuates between 0.869 and 0.874. Recall remains consistently high, fluctuating around 0.975893. For Loan, accuracy, F1, precision, and recall are all insensitive to *ntree*. Increasing the value of *ntree* does not significantly improve these metrics and the optimal value for the *ntree* hyperparameter is 5.

Extra Credit Questions

There are two ways in which you may receive extra credit in this homework.

[QE.1] Extra points (10 Points) Analyze a third dataset: the **Raisins Dataset**. The goal here is to use morphological features extracted from raisin images to classify each instance as either a Kecimen or a Besni raisin variety. The dataset contains 900 instances. Each instance is described by 7 *numerical* features, and the dataset contains two classes. You should present the same analyses and graphs discussed above.

This dataset can be found in the “*Supporting Files*” zip file for this homework on Canvas.

Questions

For each dataset, you should:

1. Train a random forest and evaluate it using the stratified cross-validation technique discussed above. You should train random forests with different values of the *ntree* parameter: 1, 5, 10, 20, 30, 40, and 50.
2. For each value of *ntree*, you should measure the accuracy, precision, recall, and F1 score of the resulting random forest.

Raisin Hyperparameter ($k = 5$, max_depth=10, min_samples_split=5)

Table 3: Raisin Result ($k = 5$, max_depth=10, min_samples_split=5)

ntrees	acc	prec	recall	f1
1	0.796667	0.805997	0.784444	0.793242
5	0.832222	0.849506	0.808889	0.828039
10	0.855556	0.889168	0.813333	0.848870
20	0.856667	0.885104	0.820000	0.851053
30	0.868889	0.897423	0.833333	0.863826
40	0.864444	0.894308	0.826667	0.858862
50	0.863333	0.896393	0.822222	0.857330

3. You should then create, for each dataset and for each of the four metrics described above, a graph that shows the value of *ntree* on the x-axis and the value of the corresponding metric on the y-axis. For example, you should include one graph showing the accuracy of the random forest as a function of *ntree* for Dataset #1, another graph showing the precision of the random forest as a function of *ntree* for Dataset #1, and similar graphs for the other metrics and for Dataset #2.

Important: The analyses mentioned above should result in a total of **8 graphs**. Please indicate clearly in the caption of each figure/graph which results are shown in that graph.

Raisin Hyperparameter ($k = 5$, max_depth=10, min_samples_split=5)

Raisin:

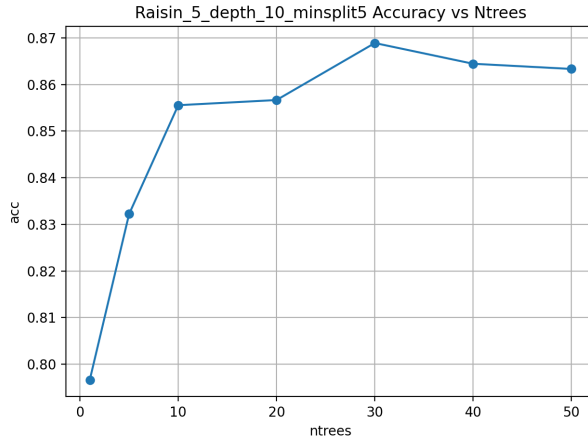


Figure 10: Accuracy for Raisin dataset.

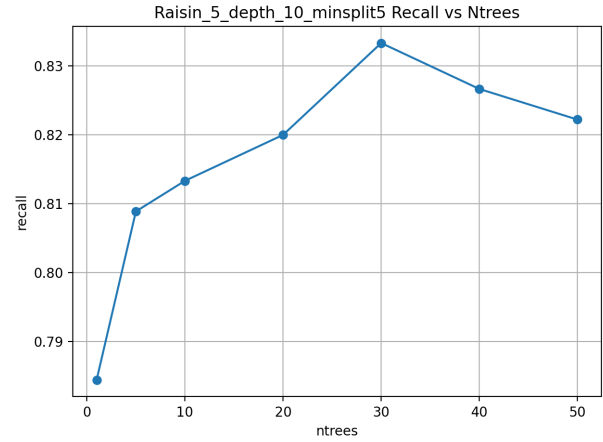


Figure 11: Recall for Raisin dataset.

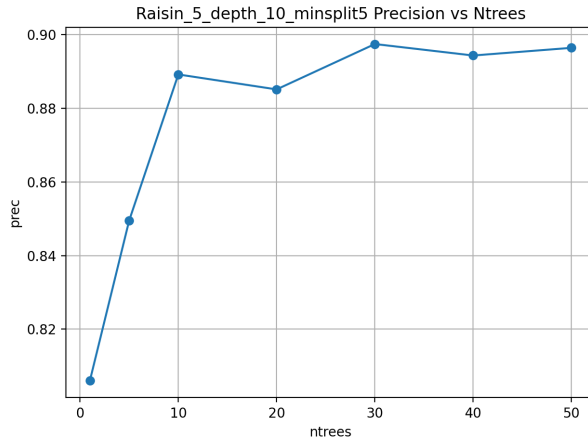


Figure 12: Precision for Raisin dataset.

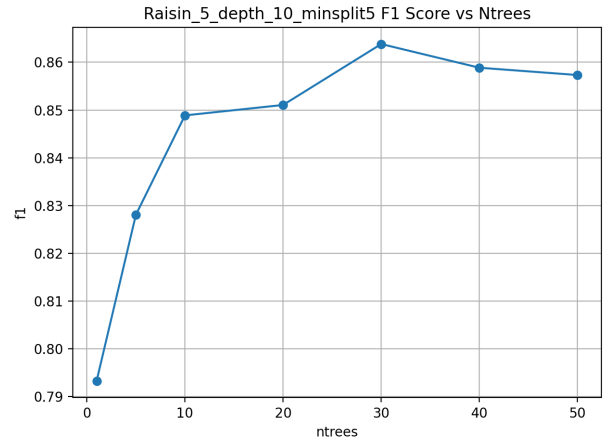


Figure 13: F1 for Raisin dataset.

4. For each metric being evaluated (and for each dataset), discuss which value of *ntree* you would select if you were to deploy this classifier in real life. Explain your reasoning.

Raisin Hyperparameter ($k = 5$, max_depth=10, min_samples_split=5)

Raisin ntree:

- Accuracy: 30
because it gives the highest accuracy.
- Precision: 30
because it gives the highest precision.
- Recall: 30
because these three values achieve the same highest recall.
- F1: 30
because it gives the highest f1.

5. Discuss, at a high level, which metrics were more directly affected by changes in the value of *ntree* and, more generally, how these changes influenced the performance of your algorithm. For example, was the accuracy of the random forest particularly sensitive to increasing *ntree* beyond a certain value? Was the F1 score a “harder” metric to optimize, possibly requiring a larger number of trees in the ensemble? Is there a point beyond which adding more trees does not improve performance (or even makes it worse)?

For Raisin, all four metrics increase as *ntree* increases, showing a monotonic rise from 1 to 30. On the Raisin dataset, accuracy, precision, recall and F1 are more sensitive to changes in *ntree*. The optimal *ntree* hyperparameter for WDBC is 30; values above this do not yield further gains and may even result in a slight decline.

[QE.2] **Extra points (10 Points)** Analyze a fourth dataset: the **Titanic Dataset**. On April 15, 1912, the largest passenger liner ever made collided with an iceberg during her maiden voyage. When the Titanic sank, it killed 1502 out of 2224 passengers and crew. One of the reasons the shipwreck resulted in such loss of life was that there were not enough lifeboats for the passengers and crew. Although luck was involved in surviving the sinking, some groups of people were more likely to survive than others. The goal here is to predict whether a given person was likely to survive this tragedy. The dataset contains data for 887 real passengers of the Titanic. Each row represents one person. The columns describe different attributes of the person, including whether they survived (the class/label), their age, their passenger class, sex, and the fare they paid. Notice that this dataset combines *both numerical and categorical attributes*. You should present the same analyses and graphs discussed above.

This dataset can be found in the “*Supporting Files*” zip file for this homework on Canvas.

Questions

For each dataset, you should:

1. Train a random forest and evaluate it using the stratified cross-validation technique discussed above. You should train random forests with different values of the *ntree* parameter: 1, 5, 10, 20, 30, 40, and 50.
2. For each value of *ntree*, you should measure the accuracy, precision, recall, and F1 score of the resulting random forest.

Titanic Hyperparameter ($k = 5$, max_depth=15, min_samples_split=2)

Table 4: Titanic Result ($k = 5$, max_depth=15, min_samples_split=2)

ntrees	acc	prec	recall	f1
1	0.758782	0.676421	0.722208	0.697703
5	0.817336	0.788254	0.725064	0.754321
10	0.817336	0.812804	0.692924	0.746419
20	0.821869	0.820099	0.695993	0.751529
30	0.815115	0.804686	0.693180	0.743208
40	0.818504	0.818334	0.684399	0.744327
50	0.817368	0.818164	0.684356	0.743630

3. You should then create, for each dataset and for each of the four metrics described above, a graph that shows the value of *ntree* on the x-axis and the value of the corresponding metric on the y-axis. For example, you should include one graph showing the accuracy of the random forest as a function of *ntree* for Dataset #1, another graph showing the precision of the random forest as a function of *ntree* for Dataset #1, and similar graphs for the other metrics and for Dataset #2.

Important: The analyses mentioned above should result in a total of **8 graphs**. Please indicate clearly in the caption of each figure/graph which results are shown in that graph.

Titanic Hyperparameter ($k = 5$, max_depth=15, min_samples_split=2)

Titanic:

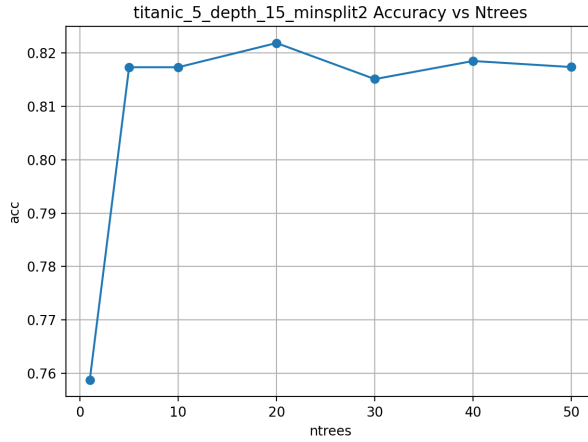


Figure 14: Accuracy for Titanic dataset.

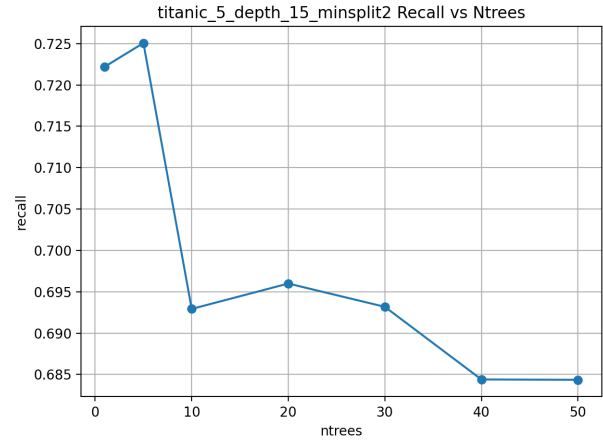


Figure 15: Recall for Titanic dataset.

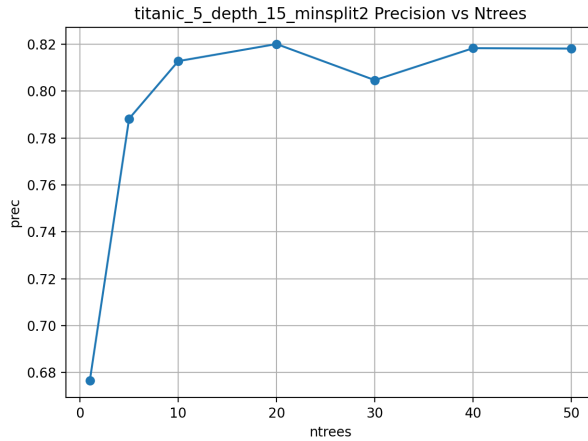


Figure 16: Precision for Titanic dataset.

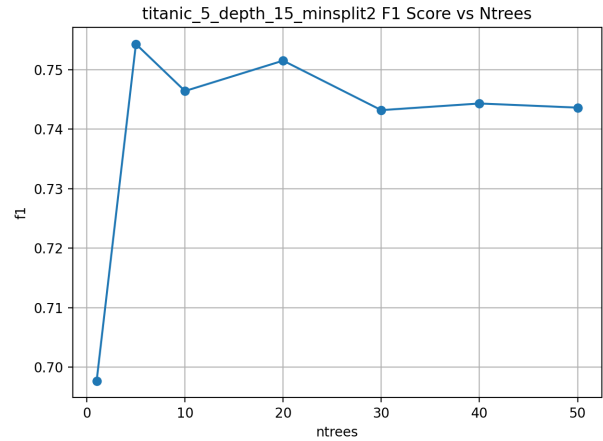


Figure 17: F1 for Titanic dataset.

4. For each metric being evaluated (and for each dataset), discuss which value of *ntree* you would select if you were to deploy this classifier in real life. Explain your reasoning.

Titanic Hyperparameter ($k = 5$, max_depth=15, min_samples_split=2)

Titanic ntree:

- Accuracy: 20
because it gives the highest accuracy.
- Precision: 20
because it gives the highest precision.
- Recall: 5
because these three values achieve the same highest recall.
- F1: 5
because it gives the highest f1.

5. Discuss, at a high level, which metrics were more directly affected by changes in the value of *ntree* and, more generally, how these changes influenced the performance of your algorithm. For example, was the accuracy of the random forest particularly sensitive to increasing *ntree* beyond a certain value? Was the F1 score a “harder” metric to optimize, possibly requiring a larger number of trees in the ensemble? Is there a point beyond which adding more trees does not improve performance (or even makes it worse)?

For Titanic, different metrics respond differently to changes in *ntree*: accuracy and precision peak at *ntree* = 20, while recall and F1 score peak at *ntree* = 5. There is no single *ntree* value that optimizes all metrics simultaneously on the Titanic dataset. Furthermore, for the Titanic dataset, increasing *ntree* from 1 to 5 or 20 yields significant improvements; however, after increasing from 20 to 30, 40, or 50, neither accuracy nor F1 score continues to rise slowly, and some metrics even decrease slightly.